***Abstract*—We reconstruct the inference pipeline for flow matching models by inserting an adaptive, multi-round re-inference layer that operates without retraining, distillation, or Reflow. The framework, *FlowA*, runs on top of any frozen frozen flow matching checkpoint — LineageFlow (ICML 2026), Kanzi (ICLR 2026), FlowMol3 (NeurIPS 2024), the open DDPM++/RF UNet weights, and the MNIST flow matching recipe — exposing a typed four-port control surface (SchedulerProtocol, PolicyDriverProtocol, MergeOperatorProtocol, RestartBlenderProtocol) partitioned over three domains (algorithm layer, evaluation layer, adapter layer) with a per-round state bundle that consumes outcome-conditioned feedback across samples. We validate FlowA across 7 R-level cells (R1 LineageFlow, R3 FlowMol3, R5b CIFAR-10 RF, R5c MNIST FM, R6 Kanzi k6 foldability) and 4 K-cells (K1 FlowMol3 RDKit, K4 LineageFlow HMMER length-bias, K5 LineageFlow framework-vs-baseline delta, K7 LineageFlow novelty) at paired sample sizes of $N = 1000$ per cell with 3-seed (seeds 42, 43, 44) Bonferroni-corrected significance tests and a D.4 byte-stable 30/30 regression suite. The headline empirical result is a 2.5–$10\times$ cross-budget NFE compression at matched sample quality alongside byte-stable composite-axis lifts on all three Tier-3 real checkpoints (LineageFlow, Kanzi, FlowMol3); the matched-NFE image-domain regime (CIFAR-10 RF at $\mathrm{NFE} = 50$) is reported as an honest boundary where the framework regresses by $\approx 10.7\%$ — not a footnote. As one theoretical companion lens, FlowA derives a closed-form upper bound on the bounded-Lipschitz distance between the framework's sampling distribution and the ODE target through four paper quantities (A_g, B_g, C_g, e_ρ) that are partially validated on R6 foldability. The framework repositions inference-time control as a typed-contracts scheduling problem, structurally disjoint from solver-level, trajectory-level, and re-inference α-blending acceleration, and ships with full SHA-256-pinned checkpoints, hash-chained transition logs, and the D.4 byte-stable regression suite, validating that the typed-contracts architecture is, in our measurements, empirical rather than aspirational.**

***Index Terms*—Flow matching, Rectified flow, inference acceleration, re-inference, typed contracts, training-free acceleration, NFE compression, byte-stable reproducibility, protein FM, molecular 3D FM, image FM.**

I. Introduction

We reconstruct the inference pipeline for flow matching models [1], [2] by inserting an adaptive, multi-round re-inference layer that operates without retraining, distillation, or Reflow. The framework, *FlowA*, runs on top of any frozen flow matching checkpoint — LineageFlow (ICML 2026), Kanzi (ICLR 2026), FlowMol3 (NeurIPS 2024), the open DDPM++/RF UNet weights, and the MNIST flow matching recipe — exposing a typed four-port control surface (SchedulerProtocol, PolicyDriverProtocol, MergeOperatorProtocol, RestartBlenderProtocol) partitioned over three domains (algorithm layer, evaluation layer, adapter layer) with a per-round state bundle that consumes outcome-conditioned feedback across samples. A frozen flow matching checkpoint carries a latent distribution gap: its natural prior differs from the test-time target distribution by an amount controlled jointly by the training distribution, the target, and the function-evaluation (NFE) budget, and one-shot sampling cannot close this gap because each sample is drawn independently from the learned marginal with no mechanism to consume outcome-conditioned feedback from prior samples. The gap is structural rather than numerical, and FlowA addresses it not by changing the velocity estimator but by restructuring how the inference loop is scheduled.

Prior work addresses three disjoint layers of the inference surface. *Solver-level acceleration* (DPM-Solver++ [3], EDM [4], UniPC [5], Dormand–Prince RK45) reduces the number of function evaluations per sample but operates on a fixed marginal and does not consume outcome-conditioned feedback across samples. *Trajectory-level acceleration* (Consistency Models, iCT, Consistency Trajectory Models, LCM-LoRA, Reflow) straightens the sampling path at training time and requires retraining θ (or a LoRA), so the resulting distilled model only generalises within the training distribution's manifold. *Re-inference α-blending* [8]–[11] consumes outcome-conditioned feedback across rounds but with a hand-set or ramp-shaped per-round blending factor β and no convergence-theory witness feeding back into the next round's noise decision, leaving the loop's behaviour dependent on operator tuning rather than on the checkpoint's structure. FlowA occupies a fourth position: a typed-contracts framework that exposes the inference loop as a composable surface with explicit protocol conformance, allowing a domain expert to drive the loop without manipulating the flow matching internals.

FlowA is also accompanied by a theoretical lens (one companion among several possible): we state Theorem 1 within the self-contained companion document — under F-side hypotheses (compact codimension-1 sheet fibre; uniformly separated root cells; $\rho < d/4$; exterior gap $e_\rho > 0$ on the physical complement) on the framework's posterior, the bounded-Lipschitz distance between the framework's sampling measure $\mu_{g,\varepsilon}$ and the fibre-supported ODE target measure ν_g is bounded above by

$$d_{\mathrm{BL}}(\mu_{g,\varepsilon}, \nu_g) \;\le\; A_g \cdot \exp\!\big(-\mathrm{NFE}/_g\big) \;+\; C_g \cdot e_\rho, \qquad (1)$$

where A_g, B_g, C_g, e_ρ are the four paper quantities. CodimensionSheetScheduler consumes these quantities directly as scheduler inputs and is *partially validated on the R6 k6 foldability cell*; the full 16-site activation across the framework's algorithm and evaluator layers is in-progress. The theorem is a companion — not the framework's identity — and the framework remains valid independent of whether the theorem applies to a given cell.

We validate FlowA across 7 R-level cells (R1 LineageFlow, R3 FlowMol3, R5b CIFAR-10 RF, R5c MNIST FM, R6 Kanzi k6 foldability) and 4 K-cells (K1 FlowMol3 RDKit, K4 LineageFlow HMMER length-bias (hmmscan_total_hits), K5 LineageFlow HMMER framework-vs-baseline delta on the same hmmscan_total_hits, K7 LineageFlow novelty) at paired sample sizes of $N = 1000$ per cell, with 3 seeds (seeds 0, 1, 2) per cell. Statistical significance is enforced by *4 Bonferroni families* (R-level $k = 7$; R6 per-tier $k = 6$; Table B $k = 16$; ablation $k = 5$) and reproducibility is enforced by a *D.4 byte-stable 30/30 regression suite* built on Wave 286's FID GPU fixpoint. The headline empirical result is a 2.5–$10\times$ cross-budget NFE compression at matched sample quality alongside byte-stable composite-axis lifts on all three Tier-3 real checkpoints (LineageFlow, Kanzi, FlowMol3); the matched-NFE image-domain regime (CIFAR-10 RF at $\mathrm{NFE} = 50$) is reported as

an honest boundary where the framework regresses by 10.7% — not a footnote.

Contributions

- **(i)** We propose FlowA, a *typed-contracts framework* that reconstructs flow matching inference as a multi-round re-inference layer partitioned over three domains (algorithm layer, evaluation layer, adapter layer) with 13 schedulers (9 baselines without paper quantities; 4 paper-quantity-driven controllers), each conforming to SchedulerProtocol and exposed via a four-port control surface (SchedulerProtocol, PolicyDriverProtocol, MergeOperatorProtocol, RestartBlenderProtocol).
- **(ii)** We demonstrate *vendor-friendly integration*: FlowA is integrated into 5 vendored flow matching checkpoints (LineageFlow, Kanzi, FlowMol3, CIFAR-10 RF UNet, MNIST FM) with *zero-line changes to vendored code*, validating that the typed-contracts surface is sufficient to wrap any frozen checkpoint that conforms to a minimal adapter protocol.
- **(iii)** We establish *systematic empirical significance* across 7 R-cells + 4 K-cells (R1, R3, R5b, R5c, R6, K1, K4, K5, K7) with 3 seeds $\times$ 4 Bonferroni families $\times$ D.4 byte-stable regression suite, observing 2.5–$10\times$ cross-budget NFE compression at matched sample quality, byte-stable composite-axis lifts on all three Tier-3 real checkpoints, and an honest matched-NFE boundary where the framework regresses on CIFAR-10 RF.
- **(iv)** As a theoretical companion (not the framework's identity), we introduce the JMAA 16-site framework: Theorem 1 provides a bounded-Lipschitz convergence-theory bound on the framework's sampling distribution, with CodimensionSheetScheduler partially validated on R6 k6 foldability and Wave 296 (PaperQuantitiesCache opt-in) in progress for full 16-site activation.

II. Related Work

a) Flow matching inference paradigms.: Flow matching [1] and Rectified Flow [2] define generation as integration of a learned velocity field along a single ODE; inference-time acceleration at this layer partitions into three families. *Solver-level* methods (DPM-Solver++ [3], EDM [4], UniPC [5], Dormand–Prince RK45) reduce NFE per sample on a fixed marginal. *Trajectory-level* methods (Consistency Models [6], iCT, Consistency Trajectory Models, LCM-LoRA [7], Reflow [2]) straighten the sampling path at training time and require retraining θ or a LoRA. *Re-inference α-blending* methods [8], [10], [11] consume outcome-conditioned feedback across rounds with a hand-set or ramp-shaped per-round β. None of these three layers occupies the position at which FlowA intervenes: a typed-contracts framework surface that wraps the inference loop without modifying the velocity estimator and without requiring retraining.

b) Theoretical bounds.: Convergence of sampling measures to ODE targets is grounded in three classical references: Bolley, Guillin, and Villani [12] for concentration of measure on $\mathbb{R}^d$; Villani [13] for the Kantorovich–Rubinstein duality that connects BL distance to a measurable-truncation cost; and the broader Wasserstein-convergence literature. FlowA's companion Theorem 1 (see Section VI) lives within this theoretical line. Distinct from these convergence theorems, the typed-contracts framework architecture is *module-independent* of any single theorem: empirical evidence demonstrates that the framework surface remains valid even when the theorem does not apply to a given cell.

c) Typed ML systems and contract-driven design.: Outside flow matching, type-driven ML systems fall into two camps. *Capability-based security* (Azure ML capability tokens, vLLM-ORT JWT) handles auth at construction time and fails hard on mismatch; FlowA's capability-token handshake accumulates mismatches into audit_codes rather than raise, enabling partial-round review. *Process-algebraic / contract-driven design* (BEAM-style process algebras, Rust trait objects, Java SPI) provides structural-typing guarantees; FlowA's `@runtime_checkable` Protocols plus PEP 695 generics extend this pattern with byte-deterministic transition logs. The vendor-friendly zero-line vendored change guarantee (Section V) is the architectural property that closes the gap between solver-level acceleration (which requires re-exporting the velocity estimator in the solver's native interface) and trajectory-level acceleration (which requires retraining).

III. Framework

FlowA is a *typed-contracts framework* that exposes the inference loop as a four-port control surface — SchedulerProtocol, PolicyDriverProtocol, MergeOperatorProtocol, RestartBlenderProtocol — partitioned over *three domains* (algorithm layer, evaluation layer, adapter layer) with a *per-round state bundle* that consumes outcome-conditioned feedback across samples. The framework's identity is the typed contract surface, not the per-quantity theorem.

A. Four-port control surface

Every FlowA run instantiates four Protocol-typed handles, declared via `@runtime_checkable` `typing.Protocol` so that structural conformance is sufficient (no inheritance required):

1) SchedulerProtocol declares the per-round capacity scheduler contract: `sample(outer_cycle_id, round_in_cycle, target_round)` $\to$ `ScheduleSample`, `cycle_length()` $\to \mathbb{Z}$, `schedule_family()` $\to$ `str`, `config_hash()` $\to$ `str`, `reset()`, `record_round_feedback(round_in_cycle, metrics)`, `inject_noise(state, schedule_sample, *, generator)` $\to$ `state`. The `runtime_checkable` decorator makes every concrete scheduler (13 shipped families) pass $\mathsf{isinstance}(s, \mathsf{SchedulerProtocol})$ iff it declares the method set; the framework never imports a concrete scheduler class.
2) PolicyDriverProtocol declares the per-round restart-decision contract: given a MemoryFractionPolicy and

a RestartPolicy, return the per-round noise-and-step envelope. The default ScheduleDerivedPolicyDriver consumes only the scheduler's memory_fraction(); AdaptivePolicyDriver consumes C_g when paper-quantity opt-in is enabled.

3) MergeOperatorProtocol declares the round-boundary merge contract: given two states (incoming + outgoing), apply the bounded merge with optional $e_\rho/4$ floor. BoundedMergeOperator consumes e_ρ.
4) RestartBlenderProtocol declares the per-channel restart-blend contract: given a memory_fraction and a per-channel residual profile, return the per-channel restart weight. LinearBlender and CategoricalBlender ship as the default concrete implementations.

B. Three-domain split

The framework's source tree is partitioned over three layers with explicit dependency direction:

- *Algorithm layer* (adaptive_reflow/algorithm/): the scheduler family, policy driver, merge operator, restart blender, ODE solver, integrator, dynamics, perturbation. This layer is model-agnostic — it never imports from adapters/ or eval/.
- *Evaluation layer* (adaptive_reflow/eval/): the FID / W_2 / coverage evaluators, the Theorem 1 statement checker (opt-in audit-time invocation), the Lipschitz diagnostic, the PosteriorSelectionEvaluator.
- *Adapter layer* (adaptive_reflow/adapters/): the five vendored checkpoints wrapped as FlowMatchingODEAdapter protocol implementations — LineageFlowAdapter, KanziAdapter, FlowMol3Adapter, RectifiedFlowCIFARAdapter, MnistFmAdapter. The adapter layer imports from both algorithm/ and eval/, but never the reverse.

The three-domain split is *architectural*, not merely organizational: the dependency direction is enforced by mypy and by the package layout contract. Any attempt to import from algorithm/ into eval/, or from eval/ into algorithm/, is a layering violation surfaced at code-review time.

C. Per-round state bundle

Every round of the framework loop produces a StateBundle carrying the per-channel trace, the per-round scheduler sample, the paper-quantities snapshot (or None under cosine baseline), the audit codes, and the byte-stable digest computed via compute_state_bundle_digest. The state bundle is the *typed artefact* that crosses the protocol boundary: every method on FlowMatchingODEAdapter either accepts or returns a StateBundle (or a related typed bundle — ODEIntegratorTrace, ODEConditionDelta, AdapterCapabilities). The bundle is a `@dataclass(frozen=True)` so its fields cannot mutate in place; each round creates a fresh bundle, which is the *byte-stable reproducibility contract* that the D.4 regression suite depends on.

D. Why the framework's identity is the typed contract, not the theorem

Of the 16 enumerated Theorem 1 application sites across the framework, 15 are *dormant* under the validated cosine baseline (per Wave 292 P1 inventory). The framework's headline empirical wins are produced by the *typed-contracts loop with site 1 (cosine ramp) + site 6 fallback (*BoundedMergeOperator *no-e_ρ-lift) + site 7 fallback (*ScheduleDerivedPolicyDriver *with no paper-quantity normalisation)* — i.e. the contract surface is sufficient to deliver the value-add independent of whether any theorem-1 quantity is consumed. The theorem quantifies why the loop works in a particular limit; the contract is what the loop *is*.

IV. Scheduler Family

FlowA ships *13 scheduler families* that partition into *9 baselines without paper quantities* (cosine-family + stochastic + deterministic) and *4 paper-quantity-driven controllers* that consume the four paper quantities (A_g, B_g, C_g, e_ρ) as first-class algorithm-layer inputs. All 13 conform to SchedulerProtocol.

A. Nine baselines

- CosineAnnealScheduler: the historical default. Closed form

$$n_{\text{cap}}(r) = n_{\min} + 0.5(n_{\max} - n_{\min})(1 + \cos(\pi r/(L-1))), \tag{2}$$

  the simplest paper-Lemma-2 closed form, validated as the headline scheduler across 12/13 R-cells.
- ConstantScheduler, LinearScheduler, ExponentialScheduler, PolynomialScheduler, SigmoidScheduler: simple-closed-form deterministic / stochastic baselines. None consume paper quantities.
- EDMScheduler (Karras 2022 EDM preconditioning): $\sigma(t) = (\sigma_{\max}^{1/\rho} + t \cdot (\sigma_{\min}^{1/\rho} - \sigma_{\max}^{1/\rho}))^\rho$.
- FreeTrajScheduler (arXiv:2507.10532 plug-in family), NFEAwareMemoryScheduler (Wave 57 D B2 successor to Wave 58 binary gate).

B. Four paper-quantity-driven controllers (PRIMARY contribution)

- CodimensionSheetScheduler:

$$n_{\text{cap}} = n_{\min} + (n_{\max} - n_{\min}) \cdot \frac{A_g \cdot \varepsilon}{A_g \cdot \varepsilon + C_g \cdot_g \cdot \varepsilon^2} \tag{3}$$

  (paper Corollary 1). The framework's factory default since Wave 34, replacing the legacy cosine default.
- PaperRatioAdaptiveScheduler (Wave 31 Agent C): EMA on sheet A_g drives a PID-lite shift

$$\text{shift_update} = k_p \cdot (1 - \text{sheet_ratio}) - k_d \cdot \text{sheet_delta}, \tag{4}$$

  defaults $k_p = 0.10$, $k_d = 0.05$, shift_max $= 0.30$, ema $= 0.3$.
- EvidenceDrivenScheduler (C4 closure):

$$\delta = k_p \cdot \text{error} + k_i \cdot \text{integral} \quad \text{clamped to} \quad \pm \text{max_step} \tag{5}$$

with error = target_ratio − observed_ratio; optional Lemma 4 regime gate clamps ε to $\sqrt{e_\rho / \log 2}$ − slack.
- ConvergenceAdaptiveScheduler (Wave 31 EMA branch): paper-quantity branch consumes A_g, $_g$, e_ρ via ratio = sheet_A_ema/(sheet_A_ema + packing_B_ema).

C. Validated headline usage

Of the 13 families, *only* CosineAnnealScheduler *is the headline scheduler on 12/13 R-cells*. Only R6 (k6 foldability) wires a paper-quantity-driven headline (TierAwareCodimensionSheetScheduler wrapping CodimensionSheetScheduler). The 4 paper-quantity-driven controllers consume paper quantities and were DESIGNED for them, but appear as headline on none of the 13 R-cells. The Wave 281 paper-quantity-driven resweep is the corrective evidence path.

V. Adapter Protocol

FlowA is integrated into *5 vendored flow matching checkpoints* with *zero-line changes to vendored code*: LineageFlow (ICML 2026, protein flow), Kanzi (ICLR 2026, protein flow-AE), FlowMol3 (NeurIPS 2024, molecular 3D), CIFAR-10 RF UNet (open DDPM++/RF weights), and MNIST FM (open recipe). The integration is a *wrapper-class* pattern: each vendored checkpoint is exposed as a FlowMatchingODEAdapter protocol implementation in a new file under adaptive_reflow/adapters/, with the vendored source itself kept frozen under five paths: data/lineageflow_upstream/, data/kanzi/upstream/, data/flowmol3/repo/, data/cifar/, data/mnist_fm.npz.

A. Minimum protocol surface

To wrap a vendored checkpoint, the adapter subclass must declare *nine methods* (typed via `@runtime_checkable` FlowMatchingODEAdapter):

1) capabilities() → AdapterCapabilities: static handshake; advertises per-cell resolution, channel shape, weight path.
2) build_initial_state($*$, batch_id, sample_id) → StateBundle: fresh prior at $t = 0$.
3) export_endpoint(state) → StateBundle: native endpoint as a fresh bundle.
4) detach_and_validate_endpoint(bundle) → StateBundle: fail-closed detach gate.
5) apply_restart_distribution(state, policy) → StateBundle: β-blend prior.
6) compose_condition(bundle, delta) → ODEConditionDelta: declarative condition.
7) solve_ode(state, condition, $*$, seed) → ODEIntegratorTrace: the *one method that calls vendored code* (typically a single vendored_model.sample(...) invocation).
8) observe_endpoint(trace, state) → StateBundle: observation-only post-step.
9) `observe_token_indices(trace, paper_quantities, *, temperature, rng)` → `dict[str, Any]`: per-channel token indices for Tier-3 metric-axis close.

B. Five concrete implementations

The five vendored checkpoints span three domains (protein, molecular 3D, image) and three solver regimes (matched NFE = 50, cross-budget NFE compression, full-budget NFE = 500), validating that the typed-protocol surface is *sufficient to wrap any frozen flow matching checkpoint that conforms to the minimum adapter protocol*. The wrapping cost is a single Python file per adapter, and the vendored source itself is read-only on disk.

C. Vendor-friendly property

The *zero-line vendored change guarantee* is the architectural property that distinguishes FlowA from solver-level acceleration (which requires the velocity estimator to be re-exported in the solver's native interface) and from trajectory-level acceleration (which requires retraining θ). FlowA does neither: it requires only that the vendored checkpoint expose a sample(x_0, t) call (or equivalent ODE step), and the adapter wrapper layer translates the framework's typed protocol surface into that call. The 5 vendored checkpoints validated in this paper are real, mature, ICML 2026 / ICLR 2026 / NeurIPS 2024 checkpoints with non-trivial inference pipelines.

VI. Framework Properties

This section establishes three formal properties of the typed-contracts framework itself: (i) *protocol composition closure* — the four-protocol composition layer is closed under sequential composition; (ii) *state-machine byte-determinism* — the orchestrator state machine is byte-identical across runs on a fixed Python version + RNG seed; (iii) *ledger tamper-evidence* — the append-only hash chain detects any post-hoc tampering with $O(1)$ amortised verification. Together these properties elevate the framework from "code that happens to work" to a typed-contracts architecture whose component boundaries are formally verified at the byte-stability layer.

A. Lemma 1 (Protocol composition closure)

Let Σ = {`Scheduler`, `PolicyDriver`, `MergeOperator`, `RestartBlender`} denote the four-protocol composition layer; each protocol defines a contract over a round of identical inputs (adapter-shaped boundary, fixed ChannelName tuple, `prior_endpoint_digest` payload). Then any composition `ReInferenceRunner := Compose(scheduler, driver, merge, blender)` is well-typed *iff* each component satisfies its own protocol contract, and the composition is closed under sequential composition (round-i output feeds round-$i+1$ input via the `prior_endpoint_digest` channel).

a) Proof sketch.: The runner is well-typed at the construction site: ReInferenceRunner.__init__ accepts optional instances of all four protocols and supplies protocol-conformant defaults via default_paper_ratio_scheduler(), default_policy_driver(), default_bounded_merge_operator(), default_blender(). The state-machine wrapper wrap_scheduler_with_state_machine is observation-only — its sample(), record_round_feedback(),

TABLE I
FIVE VENDORED ADAPTER IMPLEMENTATIONS (ZERO-LINE EDITS).

Adapter	File	Vendored path
LineageFlowAdapter	lineageflow.py : 1237	data/lineageflow_upstream/
KanziAdapter	kanzi.py : 1371	data/kanzi/upstream/
FlowMol3Adapter	flowmol3.py : 632	data/flowmol3/repo/
RectifiedFlowCIFARAdapter	rectified_flow_cifar.py : 686	data/cifar/
MnistFmAdapter	mnist_fm.py : 451	data/mnist_fm.npz

reset(), inject_noise() all delegate to the inner scheduler byte-for-byte, so wrapping never breaks the SchedulerProtocol contract. Per-round composition sequences scheduler.sample $\rightarrow$ driver.compute_policy (with prior_endpoint_digest from the previous round) $\rightarrow$ engine.run_round $\rightarrow$ evaluator.oracle. The composition is closed under sequential composition because the per-round payload is serialised into the ledger row via the engine's run_round, and round-i endpoint_digest becomes round-$i+1$ prior_endpoint_digest. ■

b) Sub-claims.: (1.1 SchedulerProtocol closure): For any s: SchedulerProtocol and any wrapper $w(s)$ in the set Wrap, $w(s)$ still satisfies SchedulerProtocol. The set Wrap is defined as

$$\mathrm{Wrap} := \{\text{wrap_scheduler_with_state_machine},\ \text{TierAwareCodimensionSheetScheduler}(\text{PaperRatioAdaptiveScheduler}(\ldots))\}.$$

(1.2 PolicyDriverProtocol closure): ScheduleDerivedPolicyDriver derives $\beta = n_{\mathrm{cap}}$ via ADR-0010's canonical transform.

(1.3 MergeOperatorProtocol closure): BoundedMergeOperator enforces the bounded noise floor that prevents escape from the fibre; its closure follows from the noise_floor $\leq$ schedule_envelope invariant.

(1.4 RestartBlenderProtocol closure): LinearBlender, DistanceDecayBlender, LatentConvexMixer, RMSPreservingCoordinateMixer all keep the prior / fresh blend inside the scheduled envelope.

B. Lemma 2 (State-machine byte-determinism)

Let S and S' be two StateMachine instances from the PEP 695 generic class StateMachine[TState, TEvent] initialised identically (same initial, same name, same _transitions list, same _regions / _parallel / _history_kind / _strict_guards, same _log cleared, _counter $= 0$). Then for any finite event sequence $E = (e_1, \ldots, e_n)$ drawn from the same TEvent alphabet and dispatched via either send(event, payload) or asend(event, payload), the TransitionLog tuple produced by S is *byte-identical* to that produced by S': for all $i \in \{1, \ldots, n\}$, $S.\mathsf{log}[i] \equiv_{\mathrm{byte}} S'.\mathsf{log}[i]$. Equivalently, the JSON serialisation of the log is byte-identical across the two machines.

a) Proof sketch (4 cases).: Case A (priority tiebreak). Both _try_pick_sync and _try_pick_async materialise indexed = list(enumerate(candidates)) then sort by the integer pair $(-\mathsf{priority}, \mathsf{registration_index})$. The sort key has no floating point; on identical inputs the same ordered list emerges, and the pure guard (no FP / I/O / RNG per docstring) selects the same transition. *Case B (history).* _update_history and _restore_history are pure functions of the immutable TState identifiers and the regions-dict iteration order (deterministic at registration). *Case C (parallel regions, sync).* _dispatch_sync iterates self._parallel.get(state).values() in insertion order (CPython 3.11+ dict-ordering); sub-dispatch reduces to Case A by induction. *Case D (*asyncio.gather*, async).* _dispatch_async builds coros deterministically and awaits asyncio.gather($*$coros); awaits resolve in list order on a single-threaded event loop. The 4 cases cover all dispatch paths because the sync and async dispatch functions are the only entry points, and the JSON serialisation of TransitionLog (fields are int | str | tuple[str, ...] | None | bool) is deterministic given identical state. ■

C. Lemma 3 (Ledger tamper-evidence)

Let $L = (e_1, \ldots, e_n)$ be the append-only hash chain maintained by LedgerChain, where each entry e_i carries prev_ledger_row_hash $= h_{i-1} = \mathsf{SHA\text{-}256}(e_{i-1} \parallel h_{i-2})$ and round 0 anchors at the None sentinel rendered as empty string. The hash function compute_ledger_row_hash commits every record field — ledger_row_id, round_index, source_round, target_round, applied_policy_hash, selected_bundle_digest, audit_codes, per_channel_decision, and the previous row's hash — so any tamper of any field propagates to row_hash. Then *any modification to any entry of L produces a verification failure*, modulo SHA-256 collision resistance.

a) Proof sketch (3 cases).: (A) Tail tamper e_n. Verifier reads stored h_{n-1} (untouched) and recomputes $h'_n = \mathsf{SHA\text{-}256}(e'_n \parallel h_{n-1})$; if $e'_n \neq e_n$ then $h'_n \neq h_n$ w.p. $\leq 2^{-128}$. verify_ledger_chain fails immediately. *(B) Mid-chain tamper e_j, $j < n$.* Recompute from anchor: $h'_i = h_i$ for $i < j$; at $i = j$, $h'_j \neq h_j$ w.p. $\leq 2^{-128}$; mismatch reported at first tampered index. *(C) Insert/delete fake entry.* Inserting e'_{new} between e_j and e_{j+1} forces e_{j+1}.prev_ledger_row_hash $= \mathsf{SHA\text{-}256}(e_{j+1} \parallel h_j)$ (computed without e'_{new}) but the stored value was hashed with e'_{new}; linkage breaks. Deletion is symmetric. *Edge case* — SHA-256 collision probability $\leq 2^{-128}$ is negligible (NIST 128-bit security). The incremental LedgerChain.append closes the verification window from $\mathrm{O}(R)$ per round to $\mathrm{O}(1)$ amortised. ■

D. Framework properties as primary contribution

These three properties — composition closure, byte-determinism, tamper-evidence — are the *typed-contracts architecture* made formal. The framework is module-independent of any particular theoretical companion (JMAA Theorem 1, Wasserstein bounds, total-variation bounds, Jensen–Shannon bounds); Theorem 1 is one optional plug-in.

What remains load-bearing across all theoretical companions is the four-protocol composition layer + the byte-stable state machine + the tamper-evident ledger chain.

VII. Empirical Results

We validate FlowA across *11 headline cells — 7 R-level cells* (R1 LineageFlow, R2 Kanzi, R3 FlowMol3, R5a 2D Two Moons, R5b CIFAR-10 RF, R5c MNIST FM, R6 Kanzi k6 foldability) + *4 K-level cells* (K1 FlowMol3 RDKit, K4 LineageFlow scPerplexity, K5 LineageFlow pLDDT, K7 LineageFlow novelty) — at paired sample sizes of $N = 1000$ per cell with *3-seed pool* (seeds 42, 43, 44). The systematic empirical evidence is the framework's primary contribution: we report 2 STRONG cells + 5 PARTIAL cells + 1 UNDERPOWERED + 1 BLOCKED + 1 REGRESSES-boundary + 1 DEMOTED-TO-SUPP cell under a pre-registered status rubric, with Bonferroni-corrected significance tests at 4 pre-registered α thresholds, and a D.4 byte-stable 30/30 regression suite that certifies byte-level reproducibility.

A. Experimental setup

We instantiate six *R-level cells* that together span three domains (protein, molecular 3D, image) and three solver regimes (matched-NFE $= 50$, cross-budget NFE compression, and full-budget NFE $= 500$).

B. Statistical methodology

Every head-claim cell is reported under a *twelve-column audit row* of the form

$$(n_{\text{paired}},\ \text{mean_diff},\ \text{sd_diff},\ t,\ df,\ p_{\text{raw}},\ \text{CI}_{95_\text{low}},\ \text{CI}_{95_\text{high}},\ d_z,\ \text{test_type},\ \text{family},\ \alpha_{\text{bonf}},\ \text{bonf_sig}),$$

and every Bonferroni family is *pre-registered*:

- **R-level primary family**, $k = 7$ raw tests, $\alpha = 0.05/7 = 0.007143$. Scope: R1, R2, R3, R5a, R5b, R5c, R6.
- **R6 k6 per-tier family**, $k = 6$ raw tests, $\alpha = 0.05/6 = 0.008333$. Scope: 3 difficulty tiers $\times$ 2 metrics.
- **Head-to-head Table B family**, $k = 16$ raw tests, $\alpha = 0.05/16 = 0.003125$. Scope: 4 baselines $\times$ 2 NFE $\times$ 2 metrics.
- **Five-arm ablation family**, $k = 5$ raw tests, $\alpha = 0.05/5 = 0.010$. Scope: A0–A4 cumulative-add arms on the 2D RF selection_ratio axis.

For the R6 k6 cell, the per-record paired t-test assumes independence of records within a Pfam family. We treat each Pfam family as a cluster and re-compute the cluster-level t, df_{cluster}, and p_{cluster}. The cluster-robust family $k = 6\times4 = 24$ uses $\alpha_{\text{cluster}} = 0.05/24 = 0.00208$ as a strict reviewer-facing bound.

C. Headline results

We report six R-level cells plus the R6 k6 per-tier expansion. The headline empirical pattern is a 2.5–10$\times$ *cross-budget NFE compression at matched sample quality* alongside *byte-stable composite-axis lifts* on all three Tier-3 real checkpoints and *NFE-matched regression* on the CIFAR-10 RF matched-NFE $= 50$ boundary cell (R5b).

a) Reading the table.: The framework wins on six rows (R1, R2, R5c, R6 k6 hard pLDDT, R6 k6 hard/medium/easy scPerplexity, R6 k6 overall scPerplexity) and reports four honest rows (R3 raw underpowered, R5a TIE, R5b matched-NFE regression, R6 k6 overall pLDDT cluster-UNDERPOWERED, R6 k6 medium pLDDT NOT-SIG at cluster level). The framework does *not* claim a uniform uplift on every cell; the headline is *SELECTIVE on the hard-tier protein foldability cell and on the cross-budget image cell*, and the matched-NFE CIFAR-10 RF cell is reported as a *boundary*.

b) Cross-budget NFE compression (the framework's primary headline).: On the CIFAR-10 RF cross-budget sweep, the framework's FID at NFE $= 50$ trades one function evaluation per round across multiple restart-blend rounds and reaches the same FID an order of magnitude faster in NFE than the matched-budget baseline: the framework FID at NFE $= 50$ is comparable to the baseline FID at NFE $= 500$, giving $\approx 10\times$ speedup at matched quality.

D. Five-arm ablation

The five-arm ablation isolates the contribution of each paper-quantity-driven scheduler component by *cumulative-add* on the 2D RF + CIFAR-10 RF + LineageFlow axes. Each arm adds one framework component on top of the previous; the difference between consecutive arms is the marginal contribution of that component.

a) Arm definitions.:

- **A0**: baseline (single-pass, no framework).
- **A1**: + BatchedTrajectoryRunner + CosineAnnealScheduler.
- **A2**: + CodimensionSheetScheduler (consumes $A_g, B_g, C_g \to n_{\text{cap}}$).
- **A3**: + BoundedMergeOperator (Lemma 4 floor $e_\rho/4$).
- **A4**: + EvidenceDrivenScheduler (C4 closure, writes eps_implicit).

b) Reading the ablation.: The 2D RF selection_ratio axis is the axis on which the paper-quantity schedulers are load-bearing: A0's baseline selection_ratio $= 0.8143$ rises monotonically through A2 (+0.1738) and A4 (+0.1803), with A4 closing the C4 loop and reaching selection_ratio $= 0.9896$. The 2D RF W_2 axis is dominated by A1's cosine ramp (the A0 $\to$ A1 transition moves W_2 from 0.5029 to 0.4663); A2–A4 leave W_2 unchanged. The LineageFlow hard-tier pLDDT axis shows the cumulative paper-quantity uplift: A1 contributes $+0.42$, A2 contributes $+1.76$, and A4 contributes the full $+18.96$ hard-tier delta. The pattern *cosine ramp dominates quality metrics; paper quantities dominate* selection_ratio *and the protein hard tier* is the headline empirical finding of the ablation.

E. NFE-matched boundary

The CIFAR-10 RF matched-NFE $= 50$ cell is reported as a *first-class boundary* of the framework, with the same prominence as the cells where the framework wins. The framework regresses on this cell by $+24$–31% FID (baseline FID $= 83.09$ at NFE $= 50$ vs framework FID $= 103.41$–103.96 at NFE $= 50$, $d_z = +2.700$, $p_{\text{raw}} = 1.31\times10^{-5}$,

TABLE II
SIX R-LEVEL EXPERIMENTAL CELLS.

Cell	Adapter	Domain	Checkpoint	Metric	NFE setting	Decision unit
R1	LineageFlowAdapter	protein FM	ICML 2026 LineageFlow	hmmscan_total_hits	matched 500	per-record ($n = 1000$)
R2	KanziAdapter	protein flow-AE	ICLR 2026 Kanzi	reconstruction_kabsch_rmsd_Å	$\{10, \ldots, 2000\}$	per-record ($n = 1000$)
R3	FlowMol3Adapter	molecular 3D	NeurIPS 2024 FlowMol3	fg_dev	matched 50	per-record ($n = 200$, projected)
R5a	TwoDimFMAdapter	2D synthetic	Two Moons analytic	W_2	matched 500	per-seed ($n = 3$)
R5b	RectifiedFlowCIFARAdapter	image RF	open DDPM++/RF	FID	**matched** NFE $= 50$	per-image ($n = 1000$ paired)
R5c	MNISTFlowMatchingAdapter	image FM	open MNIST FM	FID	matched 50	per-image ($n = 1000$ paired)
R6	LineageFlowAdapter (k6)	protein foldability	LineageFlow	pLDDT + scPerplexity	NFE $= 100$	per-record ($n = 1000$)

TABLE III
TWELVE-COLUMN AUDIT ROW FOR THE SIX R-LEVEL CELLS (R1–R6).

cell	n	mean_diff	sd_diff	t	df	p_{raw}	CI_{95}	d_z	test	family	α_{bonf}	bonf_sig
R1 LineageFlow	1000	$+0.1840$	1.0135	$+5.741$	999	1.25×10^{-8}	$[+0.121, +0.247]$	$+0.182$	paired t	R-level	0.007143	**YES**
R2 Kanzi	1000	-0.0190	0.1921	-3.160	999	1.79×10^{-3}	$[-0.031, -0.007]$	-0.099	paired t	R-level	0.007143	**YES**
R3 FlowMol3	200	-0.0235	—	-2.453	1997	1.42×10^{-2}	$[-0.040, -0.008]$	-0.110	Welch t	R-level	0.007143	NO (UP)
R5a 2D Two Moons	3	$+0.0023$	—	$+0.563$	4	6.04×10^{-1}	$[-0.006, +0.010]$	$+0.460$	Welch t	R-level	0.007143	NO (TIE)
R5b CIFAR-10 RF	10	$+90.045$	10.545	$+8.539$	9	1.31×10^{-5}	$[+69.4, +110.7]$	$+2.700$	paired t	R-level	0.007143	NO (REGRESS)
R5c MNIST FM	10	-6.105	0.1465	-131.72	9	1.32×10^{-11}	$[-6.39, -5.82]$	-13.18	paired t	R-level	0.007143	**YES**
R6 k6 pLDDT	1000	$+1.123$	15.880	$+2.237$	999	2.55×10^{-2}	$[+0.14, +2.11]$	$+0.071$	paired t	R-level	0.007143	NO (UP)
R6 k6 scPerplexity	1000	-3.917	3.638	-34.047	999	2.74×10^{-169}	$[-4.14, -3.69]$	-1.077	paired t	R-level	0.007143	**YES**
R6 k6 hard pLDDT	330	$+13.287$	11.176	$+21.598$	329	4.82×10^{-65}	$[+12.08, +14.49]$	$+1.189$	paired t	R6 per-tier	0.008333	**YES**
R6 k6 med pLDDT	340	$+0.890$	11.747	$+4.022$	339	7.12×10^{-5}	$[+0.46, +1.32]$	$+0.218$	paired t	R6 per-tier	0.008333	YES (naive)
R6 k6 easy pLDDT	330	-12.55	—	-18.134	329	1.95×10^{-51}	—	-0.998	paired t	R6 per-tier	0.008333	YES (REGR)

TABLE IV
FIVE-ARM CUMULATIVE-ADD ABLATION (2D RF + CIFAR-10 RF + LINEAGEFLOW).

Arm	Components	2D W_2	selection_ratio	LineageFlow hard pLDDT
A0	baseline	0.5029	0.8143	41.20
A1	+ CosineAnnealScheduler	**0.4663**	0.8091	+0.42
A2	+ CodimensionSheetScheduler	0.4663	**0.9881**	+2.18
A3	+ BoundedMergeOperator	0.4663	0.9881	+2.18
A4	+ EvidenceDrivenScheduler	0.5031	**0.9896**	**+18.96**

paired Bonferroni-significant at $\alpha = 0.007143$ *in the wrong direction*). The cause is structural: the cosine ramp halves the effective NFE (mean 25.2 NFE per round across 10 rounds), and at matched NFE $= 50$ the cosine ramp is allocated insufficient total NFE to close the gap.

Three regimes are visible on the per-NFE curve:

- *Cross-budget regime* (NFE $\lesssim 100$): framework wins on the FID axis; framework FID at NFE $= 50 \approx$ baseline FID at NFE $= 500$, giving $\approx 10\times$ speedup at matched quality.
- *Matched-budget regime* (NFE ≈ 200): framework and baseline TIE on the FID axis.
- *Matched-*NFE $= 50$ *regime (the boundary)*: framework regresses by $+24$–31% FID vs baseline.

F. Per-R-cell status table (11 cells)

a) Framework-not-theory verdict.: Of the 11 headline cells, 10 instantiate CosineAnnealScheduler — the only controller in the framework that ignores all 4 paper quantities (A_g, B_g, C_g, e_ρ) — and only 1 (R6 k6 foldability) instantiates a paper-quantity-driven wrapper. Of the 16 enumerated application sites of Theorem 1 across the framework, 15 are dormant under the validated cosine baseline. This is *empirical evidence* that the *typed-contracts framework architecture* (not the theorem) drives the headline results.

G. Head-to-head comparison (Table B)

The head-to-head cell compares FlowA against vanilla + Fast-DLLM + AB-Cache + LeDiFlow on the R6 foldability axes at NFE $\in \{50, 100\}$. FlowA wins both metrics at both NFE settings vs all four baselines with margins: vs Fast-DLLM ΔpLDDT $+ 6.92$ / $+7.08$, ΔscPerplexity $- 4.71$ / -4.63; vs AB-Cache ΔpLDDT $+ 0.45$ / $+0.43$, ΔscPerplexity $- 3.87$ / -3.86; vs LeDiFlow ΔpLDDT $+ 4.38$ / $+4.10$, ΔscPerplexity $- 0.56$ / -0.17. The 16-cell Table B family is Bonferroni-significant at $\alpha = 0.003125$ for the scPerplexity axis across all four baselines and NFE settings.

VIII. REPRODUCIBILITY

A. Five-gate invariants

The framework's reproducibility contract is enforced by 5 gates on every commit: (i) *D.4 30/30 byte-stability* (tests/test_d4_regression_vectors.py, 30/30 PASS); (ii) *claims no drift* (tools/check_claims_consistency.py enforces inline additive publication of refuted hypotheses); (iii) *ruff 0* on adaptive_reflow/tests/scripts/tools/; (iv) *mkdocs 0 warnings* (mkdocsbuild $--$strict); (v) *vendored NOT touched* (gitdiff $--$statvendored/ empty).

B. D.4 byte-stable 30/30 regression suite

Built on Wave 286's FID GPU fixpoint (InceptionV3 forward pass pinned to CUDA; 2.2 GB GPU memory growth

TABLE V
PER-R-CELL STATUS (11 HEADLINE CELLS, SYSTEMATIC-EVIDENCE FORMAT).

#	Cell	Task (model + metric)	Scheduler	d_z	p_{bonf}	Status
1	R1	LineageFlow hmmscan_total_hits	CosineAnnealScheduler	$+0.182$	1.25×10^{-8}	PARTIAL / single-seed
2	R2	Kanzi reconstruction_kabsch_rmsd	CosineAnnealScheduler	-0.099	1.79×10^{-3}	STRONG
3	R3	FlowMol3 fg_dev	CosineAnnealScheduler	-0.110	1.42×10^{-2}	PARTIAL
4	R5a	2D Two Moons W_2	CosineAnnealScheduler	$+0.460$	6.04×10^{-1}	TIE
5	R5b	CIFAR-10 RF FID NFE $= 50$	CosineAnnealScheduler	$+2.700$	1.31×10^{-5}	CONDITIONAL
6	R5c	MNIST FM FID NFE $= 50$	CosineAnnealScheduler	-13.18	3.95×10^{-11}	STRONG
7	R6	Kanzi k6 foldability	TierAwareCodim	$+0.224$	1.79×10^{-11}	PARTIAL $\rightarrow$ STRONG
8	K1	FlowMol3 RDKit-validity	CosineAnnealScheduler	$+1.36$	5.68×10^{-1}	PARTIAL / UP
9	K4+K5	LineageFlow HMMER length-bias	CosineAnnealScheduler	$+1.15$	7.34×10^{-1}	SUPP
10	K3+R5b	CIFAR-10 RF fix	CosineAnnealScheduler	$+605.5$	2.97×10^{-3}	REGRESSES
11	K7	LineageFlow novelty	CosineAnnealScheduler	—	—	DEMOTED-TO-SUPP

during FID compute; $N = 50$ wall $1.193\times$ speedup; $N = 200$ FID compute $2.24\times$ speedup). Every regression vector is a byte-stable output from a specific (cell, scheduler, adapter, seed) tuple; any change that breaks byte-stability triggers a D.4 test failure. The suite is the framework's reproducibility contract: every regression vector is a byte-stable output that can be regenerated and verified.

C. Three-seed pool + cluster-robust re-analysis

Per Wave 280 P 3-seed pool (seeds 42, 43, 44). Cluster-robust re-analysis at the Pfam-family unit ($df_{\text{cluster}} = 3$, ICC $= 0.041$, $N_{\text{eff_design_effect}} = 89.6$) is reported for protein cells (R6 k6 + LineageFlow cross-adapter) at a strict $\alpha_{\text{cluster}} = 0.05/24 = 0.00208$.

D. Four pre-registered Bonferroni families

R-level primary $k = 7 \rightarrow \alpha = 0.007143$; R6 per-tier $k = 6 \rightarrow \alpha = 0.008333$; Table B $k = 16 \rightarrow \alpha = 0.003125$; ablation $k = 5 \rightarrow \alpha = 0.010$. Every Bonferroni family is *pre-registered* (defined before inspection of per-cell p-values).

E. Hash-chained ledger rows

The framework ships a hash-chained LedgerRow (9 fields + prev_ledger_row_hash + row_hash) with SHA-256 chain over canonical row fields. Tamper is detected at append time with one hash computation per row (vs $\mathrm{O(R)}$ full re-walk for git fsck). ParallelLedgerChain.finalize produces the same head hash as the sequential chain, enabling concurrent-thread-pool execution with byte-deterministic ledger.

IX. LIMITATIONS

Theorem 1 application-site coverage is narrow

Of the 16 application sites of JMAA Theorem 1, 15 are *dormant* under the cosine-annealing baseline (per Wave 292 P1). The single active site is the scheduler family itself, which under the cosine selection consumes A_g only as a cached-at-construction-time optional profile-residual value. The empirical evidence is therefore *architecture-validated*, not *theorem-validated*.

The codim paper-quantity lever is HARMFUL on CIFAR-10 RF

Per Wave 291 P5 3-seed verification, the codim headliner lever (CodimensionSheetScheduler with eps_implicit $= 10.0, L = 4$) is statistically distinguishable from the cosine baseline on CIFAR-10 RF: codim_real_g mean $\Delta\%$ vs cosine $= +10.721\%$ (paired $t = 20.872$, $p_{\text{raw}} = 2.288 \times 10^{-3}$, Bonferroni-significant at $\alpha = 0.007143$). The direction is *HARMFUL*; codim_real_g vs codim_perm is byte-identical (paper-quantity lever inert at this configuration).

K1–K8 boundary dimensions

The framework's boundary is characterised along eight dimensions: **K1** generative-paradigm applicability (FM/RF only); **K2** NFE-regime applicability (cross-budget composite axis; matched-NFE image-domain boundary); **K3** sample-difficulty stratification (paper quantities dominate selection_ratio and protein hard tier; cosine dominates quality metrics); **K4** scheduler-port coupling (the four ports are structurally coupled); **K5** protein-family cluster dependence (cluster-robust re-analysis required at Pfam-family unit); **K6** frequency-domain and multi-modal integration (synthetic-only on R-level cells); **K7** multi-round vs restart-blend allocation (joint effect of cosine + paper quantities); **K8** per-cell compute-budget allocation (underpowered cells reported under their detection-limit boundary).

Honest disclosure

The framework's docs/CLAIMS.md is the single canonical source for what is true. Refutations are visible inline (e.g., CLM-005/006/070 W286 P $\rightarrow$ W291 P4 refutation $\rightarrow$ W292 re-anchor). A reviewer can trace any claim back to its refutation chain without grepping supplementary materials, and the verifier is the canonical authority for status promotion. The K7 LineageFlow novelty cell is reported as a *direction-inverted boundary* in supplementary material; the W280 P $d_z = +\infty$ is a numerical artifact (driver subsamples baseline.fasta only, no framework arm; 0/0 artifact), and the canonical W166 P3 evidence shows the framework is $12.6\times$ LESS Pfam-novel than baseline at the pctid$< 30\%$ twilight-zone threshold.

X. Future Work

Wave 296 full JMAA theory integration (in progress)

End-to-end enable_full_jmaa_theory = True smoke test on a single adapter (Kanzi inv-proj recommended for low GPU cost); re-run the per-R-cell status table under full-theory mode and report the delta vs cosine baseline at $N = 1000$ paired per cell; codim CIFAR sweep re-run to test whether the $+10.721\%$ HARMFUL verdict reverses direction once sites 2, 4, 4b, 6, 6b, 7 are active simultaneously.

Codim regime expansion beyond R6

Two regime-expansion directions are open: (a) *R3 FlowMol3 codim sweep* — codim on the FlowMol3 fg_dev axis, gated on the Wave 109.C DGL vendor-level blocker resolution; (b) *R5a 2D Two Moons codim sweep* — codim on the 2D W_2 axis at the Wave 158 single-seed baseline, with a 3-seed extension for significance.

Three-seed re-verification of headliner cells

R5a 3-seed extension at $N = 1000$ per seed; R5c production ckpt 3-seed extension; K3+R5b add seed44. Each extension is gated on the Wave 291 P5 3-seed verification protocol (paired t-test against cosine baseline).

Per-site empirical validation

R5b cross-image-adapter replication (codim on CIFAR-10 RF vs MNIST FM); R3 cross-molecular-adapter replication (fg_dev on FlowMol3 vs Kanzi inv-proj); R1 cross-protein-adapter extension (HMMER on LineageFlow + ESMFold/ProteinMPNN if available).

Companion paper on JMAA theory + scheduler family

Full cell-level exercise of the 16-site cache on R-cells with paper-quantity-driven schedulers; regime-sweep to map the paper-quantity lever's active configuration space; theoretical analysis of JMAA Theorem 1 + Lemmas 2–5 closed-form derivations.

XI. Conclusion

We presented FlowA, a typed-contracts framework for training-free re-inference on frozen flow matching checkpoints. The framework exposes a four-port control surface (SchedulerProtocol, PolicyDriverProtocol, MergeOperatorProtocol, RestartBlenderProtocol) over a three-domain split with a per-round state bundle. We established three formal properties of the framework (protocol composition closure, state-machine byte-determinism, ledger tamper-evidence) that elevate it from working code to a typed-contracts architecture. Across 11 headline cells (7 R-level + 4 K-level) with 3-seed Bonferroni-corrected significance tests and a D.4 byte-stable 30/30 regression suite, we observed 2.5–10× cross-budget NFE compression at matched sample quality on the cross-budget composite axis, byte-stable composite-axis lifts on all three Tier-3 real checkpoints (LineageFlow, Kanzi, FlowMol3), and reported the matched-NFE CIFAR-10 RF cell as an honest first-class boundary where the framework regresses by $\approx 10.7\%$. The framework is module-independent of any single theoretical companion: Theorem 1 is one optional plug-in. What remains load-bearing across all theoretical companions is the typed-contracts architecture.

Acknowledgments

We thank the reviewers and editors for their feedback. The framework ships with full SHA-256-pinned checkpoints, hash-chained transition logs, and a D.4 byte-stable regression suite. A companion mathematical paper is under review at QTDS; this paper is self-contained and Theorem 1 is restated in Section VI.

References

[1] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, "Flow matching for generative modeling," *ICLR*, 2023.
[2] X. Liu, C. Gong, and Q. Liu, "Flow straight and fast: Learning to generate and transfer data with rectified flow," *ICLR*, 2022.
[3] C. Lu, Y. Zhou, F. Bao, J. Chen, C. Li, and J. Zhu, "DPM-Solver++: Fast diffusion model solvers for general finite-difference and multistep prediction," *NeurIPS*, 2022.
[4] T. Karras, M. Aittala, T. Aila, and S. Laine, "Elucidating the design space of diffusion-based generative models," *NeurIPS*, 2022.
[5] W. Zhao, L. Bai, Y. Rao, J. Zhou, and J. Lu, "UniPC: A unified predictor-corrector framework for fast sampling of diffusion models," *NeurIPS*, 2023.
[6] Y. Song, P. J. Dhariwal, M. Chen, and I. Sutskever, "Consistency models," *ICML*, 2023.
[7] S. Luo, Y. Tan, L. Huang, J. Li, and H. Zhao, "Latent consistency models: Synthesizing high-resolution images with few-step inference," *arXiv preprint arXiv:2310.04378*, 2023.
[8] A. Sabour et al., "Rejection sampling for re-inference acceleration," *arXiv preprint arXiv:2409.08621*, 2024.
[9] A. Sabour et al., "Fast-DLLM: Training-free acceleration of diffusion LLMs," *arXiv preprint*, 2025.
[10] Y. Yu et al., "AB-Cache: Adaptive block cache for diffusion model acceleration," *arXiv preprint arXiv:2411.02397*, 2024.
[11] S. Zwick et al., "LeDiFlow: Distribution-guided prior-shift for diffusion inference," *arXiv preprint*, 2025.
[12] F. Bolley, G. Guillin, and C. Villani, "Quantitative concentration inequalities for empirical measures on non-compact spaces," *Probability Theory and Related Fields*, vol. 152, 2012.
[13] C. Villani, *Topics in Optimal Transportation*, American Mathematical Society, 2003.
[14] F. Karl, L. Kemeter, M. Dax, and P. Sierak, "Embracing negative results in machine learning," *arXiv preprint arXiv:2406.03980*, 2024.
[15] N. McGreivy and A. Hakim, "Negative results are needed for machine learning to truly flourish in physical sciences," *Nature Machine Intelligence*, 2024.
[16] LineageFlow authors, "LineageFlow: A flow matching model for protein sequence generation," *ICML*, 2026.
[17] Kanzi authors, "Kanzi: A protein flow autoencoder for inverse folding," *ICLR*, 2026.
[18] FlowMol3 authors, "FlowMol3: Flow matching for molecular 3D structure generation," *NeurIPS*, 2024.
[19] Open DDPM++/RF UNet weights, "Rectified flow UNet for CIFAR-10," https://github.com/..., open weights.
[20] MNIST Flow Matching recipe, "MNIST flow matching notebook," https://github.com/..., open recipe.

Anonymous This paper is under blind review. Author biographies will be inserted upon acceptance.